

# AWS Resource Explorer

Prove your credentials work and discover the two ways Boto3 talks to AWS.

|                           |                                 |                                                |                                        |
|---------------------------|---------------------------------|------------------------------------------------|----------------------------------------|
| DURATION<br><b>1 h 30</b> | SERVICES<br><b>STS, S3, EC2</b> | DELIVERABLE<br><b>aws_resource_explorer.py</b> | PATTERN<br><b>Clients vs Resources</b> |
|---------------------------|---------------------------------|------------------------------------------------|----------------------------------------|

## 1 Goal

Build one Python script that reports on the AWS account it is connected to: who you are, which region Boto3 resolved, which S3 buckets and EC2 instances exist, and which regions are available. When something goes wrong, the script must fail politely with a readable message instead of a traceback.

### PREVIEW OF THE OUTPUT

```
AWS RESOURCE EXPLORER
Lab 1 - Boto3 Fundamentals

WHO AM I?
Account ID : 123456789012
User ID    : AROA...:user3021
ARN        : arn:aws:sts::123456789012:assumed-role/vocla.../user3021

CURRENT REGION
Region     : us-east-1
Profile    : default

S3 BUCKETS (via client)
youssef-lab1-demo
Total: 1 bucket(s)
```

## 2 Learning objectives

- Explain what an SDK does and why Boto3 exists.
- Configure temporary AWS credentials and confirm they work.
- Create both a client and a resource, and describe how they differ.
- Read a nested Boto3 response without guessing its shape.
- Catch `NoCredentialsError` and `ClientError` and report them usefully.

## 3 Before you start

- [ ] Python 3.9 or newer is installed (`python --version`).
- [ ] A virtual environment is created and activated.
- [ ] `boto3` is installed (`pip install boto3`).
- [ ] The Learner Lab is started and shows a green indicator beside AWS.
- [ ] The three credential lines are pasted into `~/.aws/credentials`.
- [ ] `region = us-east-1` is set in `~/.aws/config`.

### Learner Lab reminder

Credentials are temporary and include a session token: copy all three lines (access key, secret key, session token). They expire when the 4-hour session ends, so calls that worked ten minutes ago may suddenly fail with `ExpiredToken`. Work in `us-east-1` for this and every other lab.

## 4 Step by step

### Build incrementally

Write one function at a time and run the script after each step. If you write all six functions before running anything, you will not know which one broke.

### Step 1: Prove the connection

Write `show_identity()`. It asks STS who AWS thinks you are and prints the account ID, user ID and ARN. STS needs no permissions, which makes it the ideal first call.

```
import boto3

def show_identity():
    sts = boto3.client("sts")
    identity = sts.get_caller_identity()
    print(f"Account ID : {identity['Account']}")
    print(f"User ID   : {identity['UserId']}")
    print(f"ARN       : {identity['Arn']}")

show_identity()
```

### Checkpoint

Your account number is printed. If not, stop here: nothing later in the lab will work.

### Step 2: Report the region

Write `show_region()`. Read the region from a `Session` instead of hardcoding it, so the script reports what Boto3 actually resolved.

```
def show_region():
    session = boto3.session.Session()
    print(f"Region  : {session.region_name}")
    print(f"Profile : {session.profile_name}")
```

### Checkpoint

The output says `us-east-1`. If it says `None`, your `~/.aws/config` is missing or misspelled.

### Step 3: List buckets, the client way

Create an S3 client and call `list_buckets()`. The response is a plain dictionary, exactly as AWS sent it.

Print each bucket's name and creation date. `CreationDate` is already a Python `datetime`, so Boto3 has done the conversion for you.

```
def list_buckets_with_client():
    s3 = boto3.client("s3")
    response = s3.list_buckets()

    for bucket in response["Buckets"]:
        created = bucket["CreationDate"].strftime("%Y-%m-%d %H:%M")
        print(f" {bucket['Name'][:<45]} created {created}")
```

### Checkpoint

Your buckets are listed, or nothing is printed if the account has none. An empty account is a valid result, not a failure.

## Step 4: List buckets, the resource way

Repeat the same task with `boto3.resource()`. The API underneath is the same, but the ergonomics differ: you iterate over objects and read attributes instead of digging through dictionaries.

Keep both versions side by side in your code and compare them. That comparison is the point of this step.

```
def list_buckets_with_resource():
    s3 = boto3.resource("s3")
    for bucket in s3.buckets.all():
        print(f" {bucket.name}")
```

|            | Client                                  | Resource                                         |
|------------|-----------------------------------------|--------------------------------------------------|
| Style      | Low level, one method per API operation | Object oriented, with attributes and collections |
| Returns    | Plain dictionaries                      | Python objects ( <code>bucket.name</code> )      |
| Coverage   | Every service and operation             | Only some services (S3, EC2, DynamoDB...)        |
| Pagination | Manual, or through paginators           | Collections page through results automatically   |
| Status     | Recommended for new code                | Feature-frozen, still supported                  |

### Checkpoint

Both functions print the same bucket names, and you can explain in one sentence why the second is shorter.

## Step 5: List EC2 instances, mind the nesting

`describe_instances()` returns instances nested inside reservations. A reservation is one launch request: launching three instances at once gives one reservation containing three instances. You therefore need a double loop.

Tags come back as a list of dictionaries, not as a dictionary. Convert them before looking anything up.

```
def list_instances():
    ec2 = boto3.client("ec2")
    response = ec2.describe_instances()

    for reservation in response["Reservations"]:
        for instance in reservation["Instances"]:
            tags = {t["Key"]: t["Value"] for t in instance.get("Tags", [])}
            print(f" {instance['InstanceId']} "
                  f"{instance['InstanceType']:<12} "
                  f"{instance['State']['Name']:<12} "
                  f"{tags.get('Name', '(no name)')}")
```

### Checkpoint

The script runs whether or not the account has instances. Writing `instance['Tags']` instead of `instance.get('Tags', [])` would crash on an untagged instance, so check that yours does not.

## Step 6: List the available regions

Call `describe_regions()` and print the region names sorted, four per line, so the output stays readable.

```
def list_regions():
    ec2 = boto3.client("ec2")
    names = sorted(r["RegionName"]
                   for r in ec2.describe_regions()["Regions"])

    for i in range(0, len(names), 4):
        print(" " + "".join(f"{n:<20}" for n in names[i:i + 4]))
```

### Checkpoint

Around thirty region names are printed in tidy columns.

## Step 7: Fail politely

Wrap `main()` in exception handling. Two exceptions matter here:

- **NoCredentialsError**: Boto3 searched every location and found no credentials.
- **ClientError**: AWS received your request and refused it.

Branch on the error **code**, never on the message. Codes are a stable contract; messages are prose that AWS may reword.

```
from botocore.exceptions import ClientError, NoCredentialsError

def main():
    try:
        show_identity()
        show_region()
        list_buckets_with_client()
        list_buckets_with_resource()
        list_instances()
        list_regions()

    except NoCredentialsError:
        print("[ERROR] No credentials found. Check ~/.aws/credentials")

    except ClientError as e:
        code = e.response["Error"]["Code"]
        print(f"[AWS ERROR] {code}: {e.response['Error']['Message']}")
        if code in ("ExpiredToken", "ExpiredTokenException"):
            print("Your session token expired. Restart the lab.")
```

### Checkpoint

Corrupt one character of your secret key in `~/.aws/credentials`, run the script and confirm you get a readable message instead of a traceback. Then restore the key.

## 5 Expected output

Your formatting may differ, but the information must be the same.

```
AWS RESOURCE EXPLORER

WHO AM I?
Account ID : 123456789012
ARN        : arn:aws:sts::123456789012:assumed-role/voclabs/user3021

CURRENT REGION
Region : us-east-1

S3 BUCKETS (via client)
youssef-lab1-demo    created 2026-09-01 10:14
Total: 1 bucket(s)

EC2 INSTANCES
No EC2 instances in this region.

AVAILABLE AWS REGIONS
af-south-1  ap-east-1    ap-northeast-1  ap-northeast-2
...
Total: 34 regions

Done.
```

## 6 Troubleshooting

| You see                                  | It means                               | Do this                                                             |
|------------------------------------------|----------------------------------------|---------------------------------------------------------------------|
| <code>NoCredentialsError</code>          | Boto3 found no credentials anywhere.   | Check the path and spelling of <code>~/.aws/credentials</code> .    |
| <code>ExpiredToken</code>                | The session token has run out.         | Press Start Lab and re-copy all three lines.                        |
| <code>NoRegionError</code>               | No region was resolved.                | Add <code>region = us-east-1</code> to <code>~/.aws/config</code> . |
| <code>KeyError: 'Tags'</code>            | The instance has no tags.              | Use <code>instance.get("Tags", [])</code> .                         |
| <code>KeyError: 'PublicIpAddress'</code> | A stopped instance has no public IP.   | Use <code>.get()</code> with a default for every optional field.    |
| <code>ModuleNotFoundError: boto3</code>  | Wrong interpreter, or venv not active. | Activate the venv, then <code>pip install boto3</code> .            |

## 7 Cleanup

This lab creates nothing, so there is nothing to destroy. Keep the habit anyway and check the Console before leaving.

- [ ] The AWS Console shows no EC2 instances you started.
- [ ] The AWS Console shows no buckets you created.
- [ ] Your script is committed to your Git repository.

## 8 Submission checklist

- [ ] `aws_resource_explorer.py` runs end to end with no traceback.
- [ ] It prints identity, region, buckets (both ways), instances and regions.
- [ ] It handles a corrupted credential gracefully, and you tested it.
- [ ] The file is committed with a meaningful message.